

Light CCI Compare 实验

代价融合与双距离 Codex Plan

方案 D 的代码级实现说明

6 个信息组合 $\times$ 2 种距离 = 12 组 SOT

本计划以当前 `scripts.zip` 为唯一代码基线。文档不再讨论方案 A、B、C，只落实已经确定的方案 D：L、E、SR 分别形成距离矩阵，独立归一化后融合成预成本，再由最终预成本选择候选边并执行稀疏半松弛 OT。计划同时保留旧基线、完整 PIJ 导出、双距离柱状图和回归验证要求。

生成日期：2026 年 7 月 13 日

项目：2026 因果涌现

文档类型：Codex 实施计划 / 开发验收依据

目录

1. Codex 必须先理解的最终语义	4
1.1 旧组合与新组合不是同一种算法	4
1.2 三类信息的含义	4
1.3 距离公式	4
1.4 距离矩阵归一化	5
1.5 最终预成本	5
1.6 候选边必须由最终融合成本决定	5
2. 当前代码核查结论	6
2.1 当前特征拼接位置	6
2.2 当前 SOT 硬编码余弦距离的位置	6
2.3 当前已有可复用基线	7
2.4 当前已有可复用的“预成本后进 SOT”入口	7
2.5 当前主方法已经有部分可复用逻辑	7
3. 实验矩阵与方法名称	7
3.1 最终 12 个柱子	8
3.2 必须继续保留但不得混入新图的旧方法	8
4. 目标代码结构	8
5. 文件级实施计划	9
5.1 新增 mignet_ce/pij/compare/distances.py	9
5.1.1 公共 API	9
5.1.2 余弦距离	10
5.1.3 欧氏距离	10
5.1.4 SR 距离	11
5.1.5 稳健归一化元数据	11
5.1.6 主方法复用	11
5.2 修改 mignet_ce/pij/compare/features.py	12

5.2.1 防止旧权重污染新实验	12
5.2.2 保留现有标准化语义	12
5.3 新增 mignet_ce/pij/compare/cost_fusion.py	13
5.3.1 核心类	13
5.3.2 权重映射	13
5.3.3 特征只构造一次	13
5.3.4 时间对与 lower/upper 选择	14
5.3.5 行数一致性检查	14
5.3.6 构造融合成本	14
5.3.7 峰值内存约束	14
5.3.8 进入 SOT	15
5.3.9 MethodResult 的特征语义	15
5.3.10 TransitionKernels 元数据	16
5.4 十个方法文件	16
5.5 修改 mignet_ce/pij/compare/common.py	17
5.6 修改 mignet_ce/pij/compare/sparse_ot.py	17
5.7 修改 mignet_ce/config.py	17
5.7.1 新方法常量	17
5.7.2 SR 依赖	18
5.7.3 新配置字段	18
5.8 修改命令行入口	18
5.9 修改 mignet_ce/pij/registry.py	19
5.10 修改 scripts/profile_lightcci_compare_matrix.py	19
5.11 新增柱状图脚本	20
5.11.1 输入	20
5.11.2 只读取这 12 个方法	20
5.11.3 分组键	21

5.11.4 排版要求	21
5.11.5 重复行处理	21
6. 输出与元数据规范	22
6.1 结果目录	22
6.2 PIJ archive	22
6.3 pair artifacts	22
7. 测试计划	23
7.1 test_compare_distances.py	23
7.2 test_compare_cost_fusion.py	23
7.3 候选边测试	24
7.4 test_compare_sparse_ot_from_cost.py	24
7.5 registry/config 测试	24
7.6 旧基线回归	25
7.7 集成 smoke test	25
7.8 可视化测试	25
8. 推荐实现顺序	26
9. Codex 不得做的事情	26
10. 完成标准	27
11. 最终算法一句话	27

代码基线：当前来源文件 `scripts.zip`。

已确定算法：只采用“方案 D”——各信息源先独立形成距离矩阵，再归一化并融合成预成本，最后执行稀疏半松弛 OT。

本次交付目标：在不破坏现有 `compare` 30 格矩阵和历史结果的前提下，新增 10 个方法，形成 6 个信息组合 × 2 种向量距离的 12 组 SOT 实验，并新增只输出分组柱状图的可视化脚本。

1. Codex 必须先理解的最终语义

1.1 旧组合与新组合不是同一种算法

当前代码中的 `compare_E_L_sot`、`compare_L_Sr_sot`、`compare_E_Sr_sot` 走的是：

各特征块标准化 → $\sqrt{\lambda}$ 缩放 → 按列拼接 → 对拼接向量只计算一次余弦距离 → SOT.

这属于 **feature concatenation**。它们必须继续保留，以保证旧结果可复现，但不得被当作本次“特征 + 表达/SR 值”的新方法，也不得进入新柱状图。

本次新增方法采用 **cost-level late fusion / 代价层晚融合**：

L、E、SR 分别计算距离 → 每个距离矩阵独立稳健归一化，
归一化距离按权重融合为 B → 由最终 B 选择候选边，
最终预成本 B → SOT → P .

1.2 三类信息的含义

- L：沿用当前 `compare` 的 Laplacian-HKS 特征。
- E：沿用当前 `compare_E_sot` 的表达表征。这里不是 `library size`、总表达量或单个平均表达值，而是现有的多维表达特征。
- Sr：沿用当前 `compare` 的单位级 SR 一维特征。`gene` 层仍使用现有“表达加权 spot SR”逻辑，不重新定义 SR。

1.3 距离公式

对 $Q \in \{L, E\}$ ：

余弦距离：

$$D_{ij}^{Q, \cos} = 1 - \frac{Q_i^\top Q_j}{\|Q_i\|_2 \|Q_j\|_2}.$$

欧氏距离：

$$D_{ij}^{Q,\text{euc}} = \|Q_i - Q_j\|_2.$$

SR 始终使用一维绝对差：

$$D_{ij}^{SR} = |r_i - r_j|.$$

不要为 SR 单独实现“余弦版本”。正的一维标量之间的余弦相似度几乎恒为 1，没有区分能力。所谓 Cosine / Euclidean 两套实验，变化的是 L 和 E 这些向量信息的距离；SR 仍是绝对差。

1.4 距离矩阵归一化

每个时间对、每一侧、每一个信息分量都独立做 5%-95% 分位数稳健归一化：

$$\hat{D}_{ij}^Q = \text{clip} \left(\frac{D_{ij}^Q - q_{0.05}(D^Q)}{q_{0.95}(D^Q) - q_{0.05}(D^Q)}, 0, 1 \right).$$

退化规则：

1. 若 $q_{95} \leq q_{05}$ ，回退到全矩阵有限值的 min-max；
2. 若 min-max 仍退化，返回全零矩阵；
3. 在元数据中记录实际使用的归一化模式，不能静默退化。

1.5 最终预成本

对启用的信息集合 Q ：

$$B_{ij}^{(m)} = \frac{\sum_{Q \in Q} \lambda_Q \hat{D}_{ij}^{Q,m}}{\sum_{Q \in Q} \lambda_Q}, \quad m \in \{\text{cos}, \text{euc}\}.$$

第一轮正式消融固定：

$$\lambda_L = \lambda_E = \lambda_{SR} = 1.$$

权重仍作为独立配置写入程序和元数据，便于零权重退化测试以及未来敏感性分析；本轮不调参。

1.6 候选边必须由最终融合成本决定

$$\mathcal{E}_{s,t} = \text{TopK}_{\text{row}}(B) \cup \text{TopK}_{\text{col}}(B).$$

禁止先按 L 或 E 选择候选边，再只在这些边上补 SR。否则 SR 无法改变候选关系，算法不再是完整的 cost fusion。

2. 当前代码核查结论

Codex 修改前必须核对以下现状，并以此作为回归基线。

2.1 当前特征拼接位置

文件: `mignet_ce/pij/compare/features.py`

关键函数: `build_compare_feature_set()`。

当前行为:

- `_build_base_feature()` 分别构造 E/N/L/Sr;
- `_standardize_base()` 或 `_standardize_pairwise_features()` 分别标准化;
- `_feature_weight()` 读取旧的 `pij_expr_weight`、`pij_sr_weight`;
- `np.hstack(parts)` 完成最终拼接;
- 元数据写入组合规则; 完整值为:

```
combination_rule = standardize_each_base_then_concat_sqrt_weighted
```

新方法不能以多个 key 调用这个函数，否则会再次进入旧拼接语义。

2.2 当前 SOT 硬编码余弦距离的位置

文件: `mignet_ce/pij/compare/common.py`

类: `ComparePijMethodBase`

方法: `_build_pair_kernel()`。

当 `pij_key == "sot"` 时，它调用:

```
run_sparse_semi_relaxed_ot(source, target, ...)
```

文件: `mignet_ce/pij/compare/sparse_ot.py`

`run_sparse_semi_relaxed_ot()` 内部固定调用 `pairwise_cosine_distance()`，因此现有命令行 `--pij-cost-metric euclidean` 不会改变 compare SOT 的距离。

2.3 当前已有可复用基线

以下两项不改算法、不改名称、不改输出路径：

- `compare_L_sot`
- `compare_E_sot`

必须做数值回归测试，修改前后的 `P`、`EI_lower`、`EI_upper`、`EI_gain` 在浮点容差内一致。

2.4 当前已有可复用的“预成本后进 SOT”入口

文件：`mignet_ce/pij/compare/sparse_ot.py`

函数：`run_sparse_semi_relaxed_ot_from_cost()`。

它已经完成：

- 根据传入的 `dense cost` 计算行向 Top-K 与列向 Top-K 候选边并取并集；
- 候选边成本缩放；
- 稀疏半松弛 OT；
- 逐行归一化生成 `PIJ`；
- 输出候选边、稀疏 `cost`、`transport`、收敛诊断。

新方法必须复用这个入口，不要另写一套 Sinkhorn/SOT。

2.5 当前主方法已经有部分可复用逻辑

文件：`mignet_ce/pij/compare/compare_main_lap_sr_spatial_sot.py`

其中已有：

- `_robust_normalize_cost()`；
- `_pairwise_euclidean()`；
- `L`、`SR`、`spatial` 独立 `cost` 后加权的 `_pre_cost()`；
- `run_sparse_semi_relaxed_ot_from_cost()` 调用。

本次应把通用距离与归一化逻辑抽到公共模块，让主方法和新消融方法共用，避免两份公式逐渐漂移。

3. 实验矩阵与方法名称

3.1 最终 12 个柱子

图上组名	距离	PIJ 方法名	实现状态
L	Cosine	compare_L_sot	现有，直接复用
L	Euclidean	compare_L_euc_sot	新增
L+E	Cosine	compare_L_E_costmix_cos_sot	新增
L+E	Euclidean	compare_L_E_costmix_euc_sot	新增
L+SR	Cosine	compare_L_Sr_costmix_cos_sot	新增
L+SR	Euclidean	compare_L_Sr_costmix_euc_sot	新增
L+E+SR	Cosine	compare_L_E_Sr_costmix_cos_sot	新增
L+E+SR	Euclidean	compare_L_E_Sr_costmix_euc_sot	新增
E	Cosine	compare_E_sot	现有，直接复用
E	Euclidean	compare_E_euc_sot	新增
E+SR	Cosine	compare_E_Sr_costmix_cos_sot	新增
E+SR	Euclidean	compare_E_Sr_costmix_euc_sot	新增

3.2 必须继续保留但不得混入新图的旧方法

- compare_E_L_sot
- compare_L_Sr_sot
- compare_E_Sr_sot
- 以及其他旧 compare concat 方法。

它们的元数据应明确标记 fusion_mode=feature_concat。新方法标记 fusion_mode=cost_mix。单特征旧基线标记 fusion_mode=single_feature_distance。

4. 目标代码结构

完成后至少新增以下文件：

```
mignet_ce/pij/compare/  
├─ distances.py  
├─ cost_fusion.py  
├─ compare_L_euc_sot.py  
├─ compare_E_euc_sot.py  
├─ compare_L_E_costmix_cos_sot.py  
└─ compare_L_E_costmix_euc_sot.py
```

```
|— compare_L_Sr_costmix_cos_sot.py
|— compare_L_Sr_costmix_euc_sot.py
|— compare_L_E_Sr_costmix_cos_sot.py
|— compare_L_E_Sr_costmix_euc_sot.py
|— compare_E_Sr_costmix_cos_sot.py
|— compare_E_Sr_costmix_euc_sot.py

scripts/
|— plot_lightcci_cost_fusion_ablation.py

tests/
|— test_compare_distances.py
|— test_compare_cost_fusion.py
|— test_compare_cost_fusion_registry.py
|— test_compare_sparse_ot_from_cost.py
|— test_plot_lightcci_cost_fusion_ablation.py
```

每个实验方法继续保持“一个方法一个 .py 文件”的现有组织方式；十个文件只声明方法名、分量集合和距离类型，公共算法只写在 `cost_fusion.py` 中。

5. 文件级实施计划

5.1 新增 `mignet_ce/pij/compare/distances.py`

5.1.1 公共 API

建议提供：

```
def pairwise_vector_distance(
    source: np.ndarray,
    target: np.ndarray,
    metric: str,
    *,
    eps: float = 1e-12,
    block_size: int = 1024,
) -> np.ndarray:
    ...

def pairwise_euclidean_distance(
    source: np.ndarray,
    target: np.ndarray,
    *,
```

```

    block_size: int = 1024,
) -> np.ndarray:
    ...

def pairwise_scalar_absolute_distance(
    source: np.ndarray,
    target: np.ndarray,
) -> np.ndarray:
    ...

def robust_normalize_cost(
    cost: np.ndarray,
    *,
    lower_percentile: float = 5.0,
    upper_percentile: float = 95.0,
    copy: bool = False,
) -> tuple[np.ndarray, dict[str, object]]:
    ...

def summarize_dense_cost(cost: np.ndarray) -> dict[str, object]:
    ...

```

5.1.2 余弦距离

余弦入口为：

```
pairwise_vector_distance(..., metric="cosine")
```

其内部必须直接复用当前的 `pairwise_cosine_distance()`。不要复制一份新实现，否则零向量规则和数值细节可能发生分叉。

5.1.3 欧氏距离

使用分块计算，避免额外构造三维广播张量：

$$\|x - y\|_2 = \sqrt{\max(\|x\|_2^2 + \|y\|_2^2 - 2x^\top y, 0)}.$$

实现要求：

- 输入必须是二维矩阵；
- source/target 特征维度必须一致；

- 空矩阵返回正确形状的零矩阵；
- 对浮点误差导致的微小负平方距离先 `maximum(..., 0)`；
- 不能使用 `scipy.spatial.distance.cdist` 作为唯一实现，以便继续控制 `block size` 和峰值内存。

5.1.4 SR 距离

接受 `(n, 1)`；若不是一列，直接报错，避免把多维向量误当 SR：

```
np.abs(source[:, 0, None] - target[None, :, 0])
```

5.1.5 稳健归一化元数据

返回的 `metadata` 至少包含：

```
raw_summary
lower_percentile
upper_percentile
q_low
q_high
fallback_min
fallback_max
normalization_mode
nonfinite_count
normalized_summary
```

`normalization_mode` 只允许：

- `quantile_5_95`
- `minmax_fallback`
- `all_zero_degenerate`
- `empty`

5.1.6 主方法复用

修改 `compare_main_lap_sr_spatial_sot.py`：

- 删除本地重复的 `_robust_normalize_cost()` 和 `_pairwise_euclidean()`；
 - 从 `distances.py` 导入；
 - 保持主方法计算结果不变；
 - 增加回归测试，确认抽取公共函数前后预成本一致。
-

5.2 修改 mignet_ce/pij/compare/features.py

5.2.1 防止旧权重污染新实验

给 `build_compare_feature_set()` 增加仅供新方法使用的关键字参数：

```
def build_compare_feature_set(
    context,
    cfg,
    feature_keys,
    *,
    apply_feature_weights: bool = True,
) -> CompareFeatureSet:
```

默认必须为 `True`，从而所有旧调用行为不变。

当 `apply_feature_weights=False` 时：

- 仍完整复用现有 E/L/Sr 构造；
- 仍完整复用现有标准化；
- 不读取 `pij_expr_weight`、`pij_sr_weight` 来缩放特征；
- 每个基础块的 `scale=1.0`；
- 元数据记录 `feature_weight_applied=False`；
- 元数据同时记录旧配置中的 `requested weight`，但明确标记为未应用。

新 `cost fusion` 基类必须分别调用：

```
build_compare_feature_set(context, cfg, ("L",), apply_feature_weights=False)
build_compare_feature_set(context, cfg, ("E",), apply_feature_weights=False)
build_compare_feature_set(context, cfg, ("Sr",), apply_feature_weights=False)
```

禁止调用：

```
build_compare_feature_set(context, cfg, ("L", "E", "Sr"))
```

5.2.2 保留现有标准化语义

不要重写 E/L/Sr 的原始构造：

- gene 表达 PCA=64 与时间对齐保持原样；
- Laplacian-HKS 保持原样；
- spot/domain SR 与 gene 表达加权 SR 保持原样；
- gene pair 的 side-specific z-score 规则保持原样；
- 非 gene pair 的 lower+upper 全时间联合 z-score 规则保持原样。

本次只改变它们如何进入 cost，不改变基础表征。

5.3 新增 mignet_ce/pij/compare/cost_fusion.py

5.3.1 核心类

```
class CompareCostFusionSotBase:
    name: str
    component_keys: tuple[str, ...]
    vector_metric: str
```

允许的 component_keys 仅为 L、E、Sr；允许的 vector_metric 仅为 cosine、euclidean。

5.3.2 权重映射

```
_COMPONENT_WEIGHT_FIELDS = {
    "L": "compare_cost_weight_l",
    "E": "compare_cost_weight_e",
    "Sr": "compare_cost_weight_sr",
}
```

要求：

- 启用分量的权重可为 0；
- 所有启用分量权重之和必须大于 0；
- 负权重立即报错；
- 第一轮默认全部为 1。

5.3.3 特征只构造一次

每个方法、每个 vertical pair context 中，每个启用的分量只构造一次，不要在每个时间对重复读 H5AD、算 PCA 或求 HKS。

```
component_feature_sets = {
    key: build_compare_feature_set(
        context,
        cfg,
        (key,),
        apply_feature_weights=False,
    )
    for key in component_keys
}
```

5.3.4 时间对与 lower/upper 选择

新增内部 helper, 行为与当前 `ComparePijMethodBase.run()` 一致:

```
def _select_source_target(feature_set, side, pair):  
    # 若未来出现 pairwise feature, 优先用 pairwise; 否则用 timewise list.
```

虽然本轮 E/L/Sr 是 timewise, 但不要把实现写死为只能 timewise。

5.3.5 行数一致性检查

同一侧、同一时间点, 不同分量必须描述同一组单位:

- 所有 source 行数一致;
- 所有 target 行数一致;
- 若不一致, 错误信息必须包含 method、organ、layer pair、time pair、side、component shapes。

5.3.6 构造融合成本

建议内部函数:

```
def _build_fused_pre_cost(  
    component_pairs: dict[str, tuple[np.ndarray, np.ndarray]],  
    vector_metric: str,  
    weights: dict[str, float],  
) -> tuple[np.ndarray, dict[str, object]]:
```

逐分量执行:

1. 计算 raw distance;
2. 保存 raw summary;
3. 原位稳健归一化;
4. 乘权重加入 accumulator;
5. 保存 normalized summary;
6. 删除当前临时分量矩阵后再算下一项。

最后除以总权重。

5.3.7 峰值内存约束

不能同时长期保存 L、E、SR 三张 dense distance matrix。

推荐实现:

- 第一张归一化矩阵直接作为 weighted accumulator;
- 后续每次只保留 fused_cost + current_component_cost;
- 因此 3 分量时的主要峰值约为两张 dense cost, 而不是四张;
- 元数据只存 summary, 不把所有 dense component cost 塞进 Python dict;
- 仅在 export_pij_topk > 0 时保留最终 B 给 top-k 诊断; 不要默认保留三张分量矩阵。

5.3.8 进入 SOT

```
ot_result = run_sparse_semi_relaxed_ot_from_cost(
    fused_pre_cost,
    epsilon=cfg.ot_epsilon,
    gamma=cfg.ot_gamma,
    max_iter=cfg.ot_max_iter,
    source_k=cfg.ot_dist_k,
    target_k=cfg.ot_sim_k,
    raw_cost_column="raw_fused_pre_cost",
    cost_source=f"cost_mix: {'+'.join(component_keys)}:{vector_metric}",
)
```

注意当前 run_sparse_semi_relaxed_ot_from_cost() 会在候选边上再做一次现有 min-max 缩放。为保证旧 SOT 行为不受影响, 本次不要删除这个步骤; 但必须在元数据中明确:

```
component_normalization = robust_5_95_before_fusion
candidate_cost_rescaling = existing_candidate_minmax
```

5.3.9 MethodResult 的特征语义

VerticalMIGNetPipeline 即使使用预计算 PIJ, 仍要求 MethodResult 中的以下字段用于 TE/DI 与 schema 导出:

```
lower_features
upper_features
```

新方法可为接口兼容生成“诊断组合特征”:

$$Z^{diag} = [\sqrt{\lambda_L} \tilde{L} \mid \sqrt{\lambda_E} \tilde{E} \mid \sqrt{\lambda_{SR}} \tilde{SR}].$$

但必须在 metadata 中写清楚:

```
transition_construction = cost_mix
method_result_feature_role = diagnostics_and_TE_DI_only
method_result_features_used_for_P = false
```

严禁在生成 PIJ 时使用这个诊断拼接向量。

5.3.10 TransitionKernels 元数据

顶层与每个 pair/side 至少记录：

```

pij_method
fusion_mode=cost_mix
vector_metric
component_keys
component_weights
component_distance_rules
component_normalization
candidate_cost_rescaling
fused_pre_cost_summary
component raw/normalized summaries
source_shape
target_shape
candidate_edges
OT convergence
row_stochastic=true

```

5.4 十个方法文件

每个文件只写一个轻量子类。例如：

```

from mignet_ce.pij.compare.cost_fusion import CompareCostFusionSotBase

class CompareLECostMixCosSotPijMethod(CompareCostFusionSotBase):
    name = "compare_L_E_costmix_cos_sot"
    component_keys = ("L", "E")
    vector_metric = "cosine"

```

其余类按实验表逐一建立。

单分量欧氏版本也使用同一基类：

```

class CompareLEucSotPijMethod(CompareCostFusionSotBase):
    name = "compare_L_euc_sot"
    component_keys = ("L",)
    vector_metric = "euclidean"

```

不要通过一个方法名再读取 `--pij-cost-metric` 动态改变算法。方法名必须唯一编码距离语义，保证输出目录自解释。

5.5 修改 `mignet_ce/pij/compare/common.py`

只做元数据增强，不改变旧计算路径：

- 单 key 旧方法：`fusion_mode=single_feature_distance`；
- 多 key 旧方法：`fusion_mode=feature_concat`；
- SOT 旧路径继续写 `cost_source=cosine_distance_on_current_compare_features`；
- 不把新方法塞进 `parse_compare_method_name()` 的旧 30 格 parser。

这样历史 `compare_E_L_sot` 等输出仍可复现，但不会再与 `cost mix` 混淆。

5.6 修改 `mignet_ce/pij/compare/sparse_ot.py`

原则：不重写优化器，不改变现有默认行为。

必须修正的小问题：当 `dense cost` 为空时，`candidate_edges` 的 `raw cost` 列目前硬编码为：

```
raw_cosine_distance
```

应改成使用传入的：

```
raw_cost_column
```

否则新方法在空矩阵情况下会写出错误的诊断列名。

另外增加输入检查：

- `dense cost` 必须二维；
- 允许空 `shape`，但返回列名正确；
- 若存在负的有限 `cost`，直接报错，不要把它悄悄送入指数核；
- `candidate raw cost` 与 `normalized cost` 均必须有限；
- `convergence` 中写入 `raw_cost_column`。

不要修改 `_topk_candidates()` 的行向与列向并集语义。

5.7 修改 `mignet_ce/config.py`

5.7.1 新方法常量

新增：

```
COST_FUSION_NEW_PIJ_METHODS = (...10 新方法...)

COST_FUSION_EXPERIMENT_PIJ_METHODS = (
    "compare_L_sot",
    "compare_L_euc_sot",
    ...,
    "compare_E_sot",
    "compare_E_euc_sot",
    ...,
)
```

将 10 个新方法加入 PIJ_METHODS。

新增 preset:

```
"lightcci_cost_fusion": COST_FUSION_EXPERIMENT_PIJ_METHODS
"lightcci_cost_fusion_new_only": COST_FUSION_NEW_PIJ_METHODS
```

不要改变现有 lightcci_compare_matrix 的 30 格含义。

5.7.2 SR 依赖

将所有名称中包含 Sr_costmix 的新方法加入:

```
DEVELOPMENT_PIJ_METHODS
```

这样在缺少 development_feature_root 时，程序会在运行前报错。

5.7.3 新配置字段

在 TemporalRunConfig 新增:

```
compare_cost_weight_l: float = 1.0
compare_cost_weight_e: float = 1.0
compare_cost_weight_sr: float = 1.0
```

验证：三者必须非负。活跃方法的权重和检查由 cost fusion 基类执行。

现有 pij_expr_weight、pij_sr_weight、pij_graph_energy_weight 继续服务旧方法和主方法，新消融不能偷偷复用它们。

5.8 修改命令行入口

同时修改:

- `scripts/run_mignet_vertical.py`
- `scripts/run_mignet_vertical_ablation.py`

新增参数：

```
--compare-cost-weight-l 1.0
--compare-cost-weight-e 1.0
--compare-cost-weight-sr 1.0
```

并传入 `TemporalRunConfig`。

保留 `--pij-cost-metric`，因为其他旧 OT 方法可能仍使用它；但帮助文字应注明：新 `cost fusion compare` 方法的 `metric` 由方法名中的 `cos/euc` 决定，不读取这个参数。

每个方法必须继续支持单独运行：

```
python scripts/run_mignet_vertical.py \
... \
--network-method light_cci \
--pij-method compare_L_E_costmix_cos_sot \
--compare-cost-weight-l 1.0 \
--compare-cost-weight-e 1.0 \
--compare-cost-weight-sr 1.0
```

不要要求用户必须用 `ablation runner` 才能运行新方法。

5.9 修改 `mignet_ce/pij/registry.py`

- 导入十个新类；
- 在 `PIJ_METHOD_REGISTRY` 中逐一注册；
- 方法名必须与 `config` 常量完全一致；
- `registry` 测试必须断言：

```
set(COST_FUSION_NEW_PIJ_METHODS) <= set(PIJ_METHOD_REGISTRY)
```

5.10 修改 `scripts/profile_lightcci_compare_matrix.py`

新增对 `cost fusion` 方法的 `profile` 信息：

- 当前方法的 `component count`；

- vector metric;
- 每个时间对 dense cost entries;
- 一张 float64 dense cost 的 GiB;
- sequential accumulation 的预计峰值: 约两张 dense cost;
- 若开启 top-k dense diagnostics, 额外内存提醒;
- candidate edge 数量估计仍使用 $|S|K_s + |T|K_t$ 上界。

不要把 3 分量方法错误估计为长期同时持有 4 张矩阵; 实现要求是顺序累积。

5.11 新增柱状图脚本

文件: scripts/plot_lightcci_cost_fusion_ablation.py

5.11.1 输入

```
--result-root PATH
--output-dir PATH
```

result-root 支持以下结构:

```
network=light_cci/
├─ pij=<method_name>/
│  └─ metrics.csv
```

能够接受用户本机路径:

```
C:\Users\27139\Desktop\Huge Workplace\2025 Causality\output\network=light_cci
```

5.11.2 只读取这 12 个方法

脚本内建立固定 mapping, 不使用模糊字符串匹配:

```
METHOD_MAP = {
    "compare_L_sot": ("L", "Cosine"),
    "compare_L_euc_sot": ("L", "Euclidean"),
    ...
}
```

旧的 concat 方法即使目录存在, 也绝不能被读入。

5.11.3 分组键

每张图由以下四项唯一确定：

```
organ
time_pair
lower_layer
upper_layer
```

横轴固定顺序：

```
L, L+E, L+SR, L+E+SR, E, E+SR
```

每组两根柱：

- 左：Cosine；
- 右：Euclidean。

纵轴：EI_gain。

5.11.4 排版要求

沿用此前用户确认满意的柱状图排版：

- 清晰的零水平线；
- 图例只包含 Cosine、Euclidean；
- 标题包含 organ、time pair、layer pair；
- 横轴标签不截断；
- 正负柱都可见；
- PNG 至少 200 dpi；
- 缺失值留空或标 NA，禁止补 0；
- 默认只输出 PNG，不额外生成报告、汇总表或 PDF。

完整运行后应输出 36 张图：

1 organ × 6 time pairs × 6 layer pairs = 36.

5.11.5 重复行处理

正常情况下，每个 method × group 应只有一条 EI_gain。若发现重复：

- 默认报错并列出重复 key；
- 不要静默取第一条；

- 可提供显式 `--duplicate-policy mean`，但默认仍为 `error`。
-

6. 输出与元数据规范

6.1 结果目录

保持现有 ablation 结构：

```
<output_root>/
└─ network=light_cci/
   └─ pij=<new_method_name>/
      ├── metrics.csv
      ├── run_config.json
      ├── run_summary.csv
      └── progress.jsonl
```

6.2 PIJ archive

继续兼容：

```
<pij_archive_root>/
└─ network=light_cci/
   └─ pij=<new_method_name>/
      └─ organ=.../pair=.../
         ├── 11.5_to_12.5_lower_P.npz
         ├── 11.5_to_12.5_upper_P.npz
         └─ units/...
```

保持 `--export-pij --export-pij-topk 0` 可导出完整稀疏 PIJ，而不是 top-k 近似。

6.3 pair artifacts

`export_compare_pair_artifacts()` 继续输出：

- `metadata.json`
- `cost_or_kernel_diagnostics.json`
- `candidate_edges.parquet`
- `cost_sparse.npz`
- `pij_transport_sparse.npz`
- `source_mass_diagnostics.csv`
- `ot_convergence.json`

新 metadata 中必须能回答：

1. 这是 concat 还是 cost mix;
 2. 用了哪些分量;
 3. L/E 用 cosine 还是 euclidean;
 4. SR 使用什么距离;
 5. 每个分量归一化前后范围;
 6. 权重是多少;
 7. 最终 B 的范围;
 8. 候选边数和 OT 是否收敛。
-

7. 测试计划

7.1 test_compare_distances.py

至少覆盖：

1. cosine wrapper 与当前 pairwise_cosine_distance() 完全一致;
2. 欧氏距离手算矩阵;
3. 同方向不同模长: cosine=0, euclidean>0;
4. SR 绝对差手算;
5. SR 输入多列时抛错;
6. 5-95 分位归一化;
7. min-max fallback;
8. 全常数矩阵返回全零并标记退化;
9. 空矩阵保持 shape;
10. NaN/Inf 诊断正确。

7.2 test_compare_cost_fusion.py

用很小的人工矩阵手算：

$$B = (\hat{D}^L + \hat{D}^E + \hat{D}^{SR})/3.$$

断言：

- 代码与手算一致;

- 权重和归一化正确;
- $E_{weight}=0$ 时 $L+E$ 退化为 L cost;
- $E=0$, $SR=0$ 时 $L+E+SR$ 退化为 L cost;
- component row mismatch 会报清晰错误;
- cost fusion 路径从未调用多 key 的 `build_compare_feature_set()`;
- diagnostic concat 没有被用于 P 。

7.3 候选边测试

构造一个例子, 使 L 最邻近与融合后最邻近不同。断言:

- 只用 L 时候选边为 A ;
- 加 SR 后候选边改变为 B ;
- 最终候选边由 fused pre-cost 产生。

7.4 test_compare_sparse_ot_from_cost.py

覆盖:

- 自定义 `raw_cost_column` 在空矩阵时仍保留正确列名;
- $P \geq 0$;
- 非空行和为 1;
- 无 NaN/Inf;
- 负 cost 抛错;
- convergence 中记录 cost source。

7.5 registry/config 测试

- 10 个新方法都在 `PIJ_METHODS`;
- 都在 `PIJ_METHOD_REGISTRY`;
- SR costmix 无 developmental root 时 config validate 失败;
- 三个新权重不能为负;
- `lightcci_compare_matrix` 仍是原来的 30 格;
- `lightcci_cost_fusion` 恰好是 12 个方法。

7.6 旧基线回归

在同一小样本和同一配置上运行修改前/后的：

- compare_L_sot
- compare_E_sot

至少比较：

```
lower P
upper P
EI_lower
EI_upper
EI_gain
```

建议容差：

```
np.allclose(before, after, rtol=1e-10, atol=1e-12)
```

若底层 eigensolver/PCA 存在环境级微小差异，可放宽到 `rtol=1e-8`，`atol=1e-10`，但必须记录原因。

7.7 集成 smoke test

先只跑：

```
organ=heart
time points=11.5, 12.5
layer pair=seurat_k40:seurat_k150
```

至少覆盖：

- L cosine 旧基线；
- L euclidean；
- L+E+SR cosine；
- L+E+SR euclidean；
- E cosine 旧基线；
- E+SR euclidean。

确认每个方法写出一条 metrics、完整 PIJ 和 metadata 后，再扩到全矩阵。

7.8 可视化测试

构造临时 12-method metrics.csv：

- 成功生成一张 6×2 柱状图;
 - 顺序固定;
 - 旧 concat 目录不进入;
 - 缺失 Euclidean 时不补 0;
 - 重复 key 默认报错;
 - 完整真实结果应生成 36 张 PNG。
-

8. 推荐实现顺序

1. 先新增 `distances.py` 和单元测试;
2. 把主方法的重复距离/归一化 helper 切到公共模块, 并做回归;
3. 给 `features.py` 增加 `apply_feature_weights=False`, 确认旧默认不变;
4. 实现 `cost_fusion.py` 的纯 cost 构造函数及人工矩阵测试;
5. 接入 SOT、TransitionKernels 和 artifacts;
6. 新增两个单分量欧氏方法并 smoke test;
7. 新增 8 个多分量 costmix 方法;
8. 更新 config、CLI、registry、profile;
9. 增加旧方法 fusion metadata;
10. 新增柱状图脚本;
11. 跑完整 pytest;
12. 跑小样本集成测试;
13. 最后再运行 12 个正式方法。

不要一上来同时改 10 个方法文件和 SOT 内核, 否则出错时难以定位。

9. Codex 不得做的事情

1. 不得删除、改名或覆盖旧 `compare_E_L_sot`、`compare_L_Sr_sot`、`compare_E_Sr_sot`。
2. 不得把新 costmix 方法重新实现成特征拼接。
3. 不得把 SR 当作正标量做余弦距离。
4. 不得先按单一分量选候选边, 再补其他 cost。
5. 不得让新方法继承旧的 `pij_sr_weight=0.5`、`pij_graph_energy_weight=0.2`。
6. 不得让同一个方法名根据 `--pij-cost-metric` 输出两种算法。
7. 不得伪造缺失结果为 `EI_gain=0`。

8. 不得让绘图脚本扫描并自动猜测旧 `concat` 方法。
 9. 不得另写一套 OT 优化器。
 10. 不得默认同时保存三张完整 `dense component cost`，造成不必要的内存峰值。
 11. 不得改变 `compare_L_sot` 和 `compare_E_sot` 的数值结果。
 12. 不得让新增代码依赖本次报告文件或外部绘图脚本。
-

10. 完成标准

只有同时满足以下条件，任务才算完成：

- ☐ 10 个新方法可通过 `run_mignet_vertical.py --pij-method ...` 单独运行；
 - ☐ 12-method preset 正确；
 - ☐ 新方法确实是独立 `distance` → `robust normalize` → `cost fusion` → `final-B candidates` → `SOT`；
 - ☐ Cosine/Euclidean 方法名与实际距离一致；
 - ☐ SR 始终是绝对差；
 - ☐ 新权重默认为 1/1/1，旧权重没有渗入；
 - ☐ 旧 `concat` 方法保留且元数据标识清楚；
 - ☐ `compare_L_sot`、`compare_E_sot` 回归通过；
 - ☐ PIJ 非负、行随机、无非有限值；
 - ☐ `artifacts` 可追溯每个 `cost` 分量与归一化；
 - ☐ `profile` 给出新方法的内存估计；
 - ☐ 绘图脚本只输出目标柱状图；
 - ☐ 完整结果生成 36 张图，每张 6 组 × 2 柱；
 - ☐ 缺失结果不会被补 0；
 - ☐ 全部新增单元测试和 `smoke test` 通过。
-

11. 最终算法一句话

对每个 `organ`、`time pair`、`layer side`，分别从 L、E、SR 形成跨时间距离矩阵；对各矩阵独立稳健归一化并按等权融合成最终预成本 B；由 B 同时决定候选边和 SOT 传输，得到 PIJ；随后比较 lower/upper 的有效信息，并在每张图中用 Cosine 与 Euclidean 两根柱对照同一信息组合的 `EI_gain`。